

Reviewer Pack — Free Cumulant Gap in Read-Twice BP for IP_2

Entry adf4036d951b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 01:28:43 UTC

Free Cumulant Gap in Read-Twice BP for IP_2

Entry ID: adf4036d951b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 01:28:43 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For any read-twice branching program P computing IP_2 , the free cumulant κ_4 of its adjacency matrix satisfies $\kappa_4(P) \geq \Omega(n)$, whereas for read-once BPs computing IP_2 , $\kappa_4(P) = O(\log n)$.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative dependencies in branching programs. IP_2 's read-once lower bound stems from its high correlation with parity functions, which manifest as divergent free cumulants in non-commutative models. Read-twice BPs may exhibit structured dependencies that collapse cumulant growth.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f538441f45633a5a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_branching_program(n, read_twice=False):
    if read_twice:
        # Generate a read-twice branching program for IP_2
        bp = [[random.choice([0, 1]) for _ in range(2)] for _ in range(n)]
        return bp
    else:
        # Generate a read-once branching program for IP_2
        bp = [[random.choice([0, 1]) for _ in range(1)] for _ in range(n)]
        return bp

def adjacency_matrix(bp):
    n = len(bp)
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        if bp[i][0] == 1:
            A[0][i] = 1
        if len(bp[i]) > 1 and bp[i][1] == 1:
            A[i + 1][i] = 1
    return A

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def r_transform(A):
    n = len(A)
    R = [[0] * (n + 1) for _ in range(n + 1)]
    R[0][0] = 1
    for k in range(1, n + 1):
        R[k][0] = determinant(A ** k)
        for j in range(1, k + 1):
            R[j][k] = R[j-1][k-1] - R[j][k-1]
```

```

    return R

def free_cumulant(R):
    n = len(R) - 1
    kappa_4 = (R[2][3] * R[1][2] - R[2][2] * R[1][3]) / (R[0][2] * R[1][1] - R[0][1] * R[1][2])
    return kappa_4

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in range(5, 41):
        for _ in range(3): # Ensure at least 30 instances per seed
            if n == 5 and len(results) >= 27:
                break
            bp_type = 'read_twice' if random.choice([True, False]) else 'read_once'
            bp = generate_branching_program(n, read_twice=(bp_type == 'read_twice'))
            A = adjacency_matrix(bp)
            kappa_4 = free_cumulant(r_transform(A))
            results.append(kappa_4)

    mean_kappa_4 = sum(results) / len(results)
    std_kappa_4 = math.sqrt(sum((x - mean_kappa_4) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for kappa in results if (kappa >= n / 10 if bp_type == 'read_twice' else kappa >= n / 10)) / len(results)

    conjecture_holds = support_fraction >= 0.8
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "free_cumulant_kappa_4",
        "metric_value": mean_kappa_4,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 31)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}} ")
        results.append(result["metric_value"])

    mean_kappa_4 = sum(results) / len(results)
    std_kappa_4 = math.sqrt(sum((x - mean_kappa_4) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for kappa in results if kappa >= n / 10) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_kappa_4} std={std_kappa_4} support_fraction={support_fraction}")
    else:
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed=1")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ef5c7d1.py", line 97, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ef5c7d1.py", line 73, in run_trial
    kappa_4 = free_cumulant(r_transform(A))
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ef5c7d1.py", line 53, in r_transform
    R[k][0] = determinant(A ** k)
                      ~~~~~
TypeError: unsupported operand type(s) for ** or pow(): 'list' and 'int'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError, preventing data collection. Support fraction cannot be evaluated without successful runs. | next: Fix the r_transform function's matrix exponentiation implementation to handle list-based matrices properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	40575
2	preregistration	ollama_remote	qwen3:8b	0	0	19965
3	novelty	ollama_remote	qwen3:8b	0	0	16568
4	novelty	ollama_remote	qwen3:8b	0	0	9880
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21172
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11993
7	judge	ollama_remote	qwen3:8b	0	0	11293

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131445 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/adf4036d951b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/adf4036d951b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/adf4036d951b.tar.gz` (if generated)